

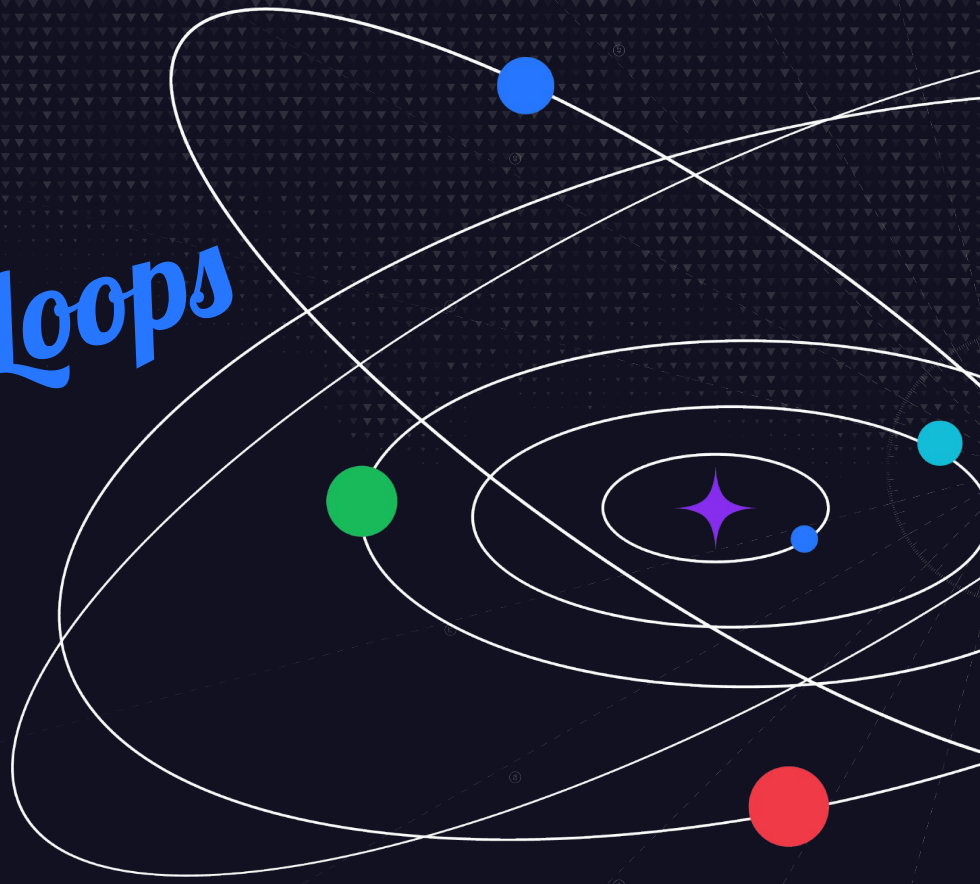
Dynamic Graphs

& Loops

Airflow for agentic
workflows

or: "Dags that choose their
own adventure"

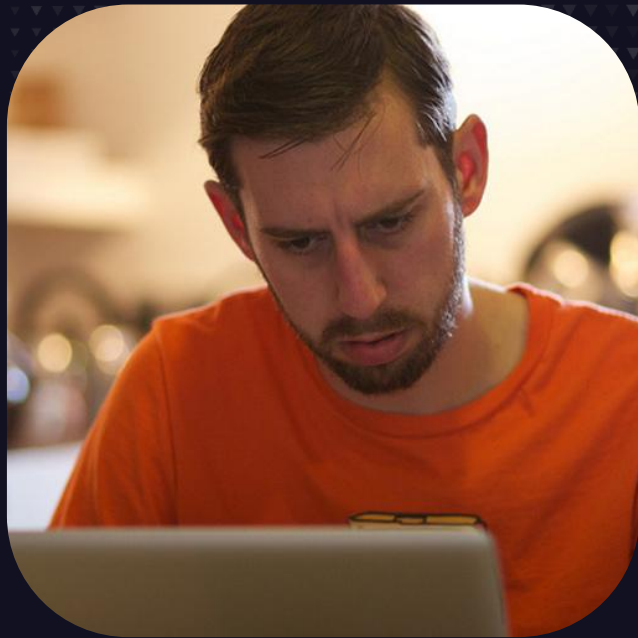
ASTRONOMER



Ash Berlin-Taylor

PMC member @ Apache Airflow
Director, Airflow Engineering @ Astronomer

ASTRONOMER





Do you like Dega?

Workloads are Changing; Airflow is too

As Vikram mentioned

1: Support richer execution models

WHAT WE EXECUTE



HOW WE EXECUTE



Richer work. More runtime dynamism. Still orchestrated.

Falsehoods Airflow believed about your Dags

Workflows are only slowly changing

Dags cannot contain cycles

Dependencies must be static and pre-defined

Falsehoods Airflow believed about your Dags

~~Workflows are only slowly changing~~

Dag versioning support

Since 3.0

And was already untrue with
Dynamic Task Mapping.

Dags cannot contain cycles

Dependencies must be static and pre-defined

First, there was Dynamic Task Mapping (2022)

Repeat a task (or a whole group), per value of upstream task.

Made "**how many**" dynamic. Left the **shape** static.

- Same tasks every index
- Count fixed once expanded
- Instances are peers — no edges, no carried state

Good for: "S3 list prefix -> process one file per task"

Doesn't help with: any "until" condition — DTM only does "for each."

Soon we'll have loops!

AIP-111: accepted & in development

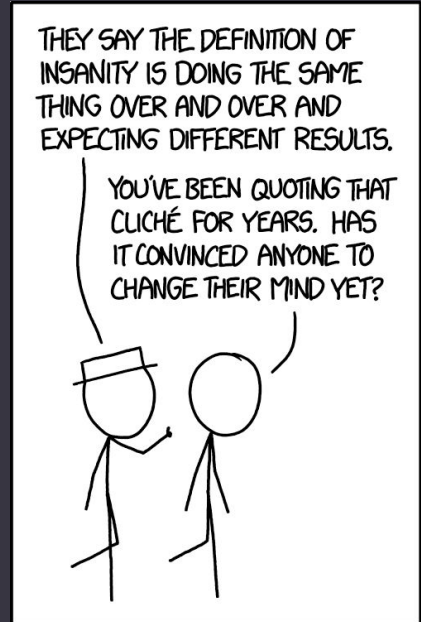
Loops? But aren't cycles forbidden?

You can't have arbitrary cycles.

But a task (group) can repeat.

Acyclic at definition time; cyclic at runtime.

Analogous to a "do while" loop



Example use case: Nightly re-training of an ML model

1. Get yesterday's labeled data.
2. Set starting weights w_0 as previous good weights
3. Repeat: find the gradient of the loss for a batch, then move downhill — $w_{n+1} = w - \eta \nabla L(w)$
4. Stop when the validation loss stops getting better for N rounds (early stopping), or when the gradient norm is very small.
5. Save the model if it is better than the current one.

Demo: Gradient Descent

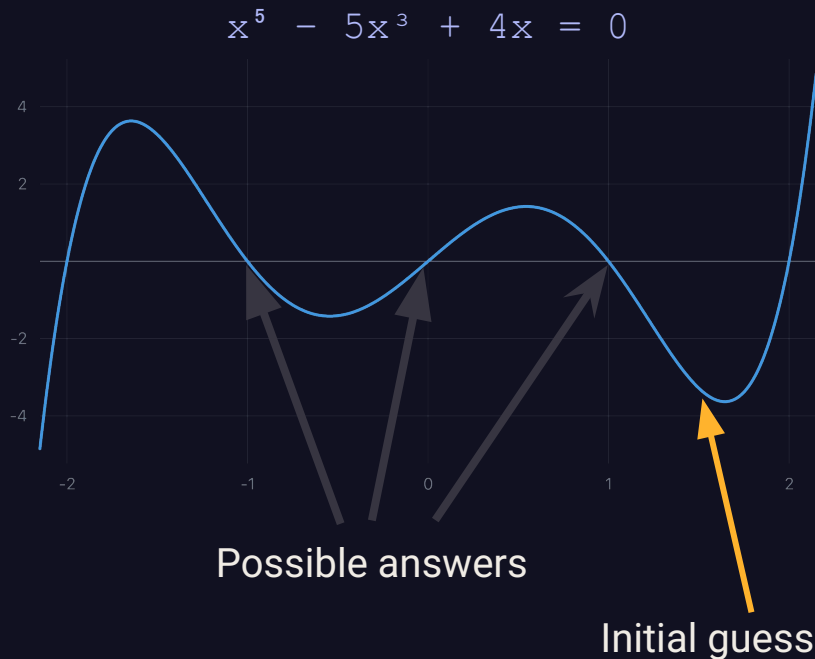
A.K.A. Newton-Raphson method

Don't worry, no maths needed!

Iterative process to find where $f(x) = 0$

Pick an initial guess, then:

- look at gradient at current point
- move in direction of negative gradient
- stop if error is small enough
- set current point to new estimate



DEMO

4 Task Instances






Dag Run
manual__2026-08-29T20:50:57.305230+00:00

▶ Trigger

Queued

↺ ✓ / ✗ 🗑

Dag Version(s)

► Manual

 Add a note...

Details

4 Task Instances

≡ Add Filter

Try Number

0



 / 


0

0

 /

0

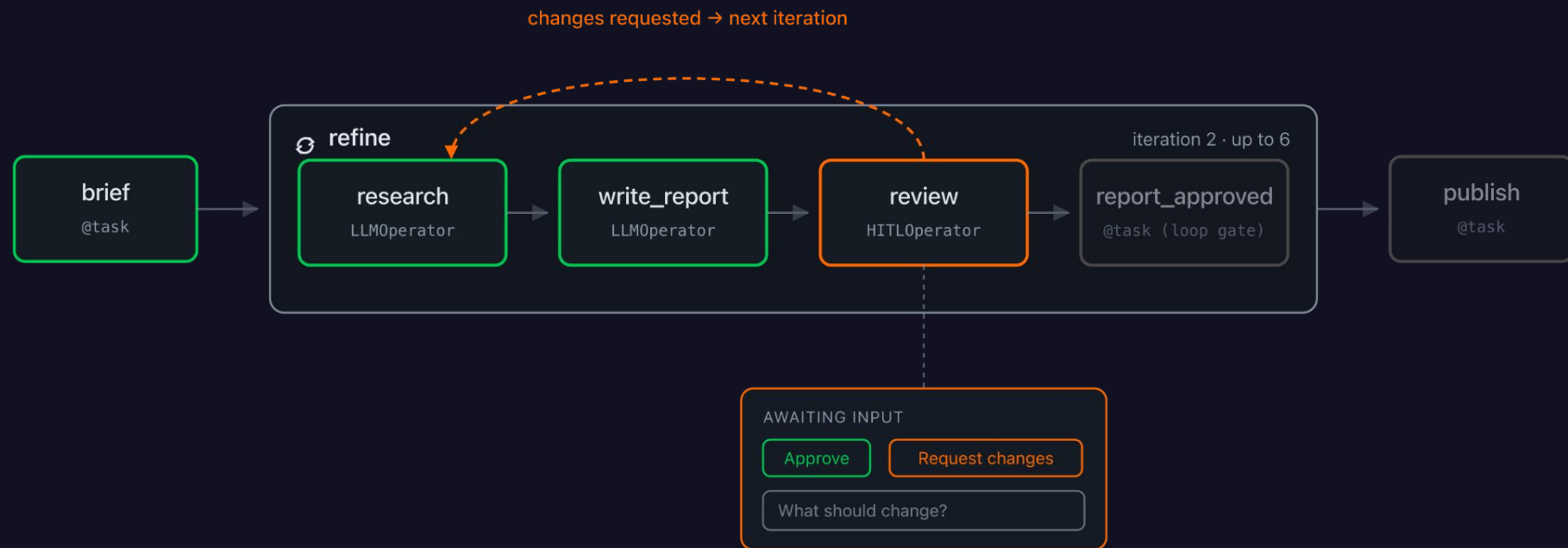






Example use case: HITL to evolve a prompt

Have a HITL Operator as final task, either accept, or feed back additional prompt to next round.



5 second syntax tour

`.loop()` on a TaskGroup

Required arguments:

- `max_iterations`
- `until` - function that returns `True` to stop loop;
- New context key: `loop`, use it to access previous result.

```
def at_final_level(loop, **context):
    return loop.result["done"]

@task_group
def body():
    @task
    def populate(loop, **context): ...
        # State carry-over: iteration N builds on iteration N-1's output (loop.previous);
        # the first iteration seeds from the upstream `seed` task.
        prev = loop.previous or ti.xcom_pull(task_ids=SEED_TASK_ID)
        depth = prev["depth"]
        ...
        return {"depth": depth}

    def decide(populated):
        done = (loop.index + 1) >= populated["depth"]
        return {**populated, "done": done}

    decide(populate())

body.loop(until=at_final_level, max_iterations=10)
```

Falsehoods Airflow believed about your Dags

~~Workflows are only slowly changing~~

Dag versioning support

Since 3.0

~~Dags cannot contain cycles~~

Task group Loops

Expected in Airflow 3.4

Dependencies must be static and pre-defined

Dynamic Execution Graphs

AIP-113 is in draft & under discussion

Reason, Act, (Evaluate,) Loop

"I can already do that"

Workarounds exist in Dags today, but all workarounds are sub-optimal

- Pre-define "every" possible path in Dag ← Noisy, can't easily see path taken
- Just run it in one task ← Bad for logs, retries and audit
- Dynamic dag file generates different layout as needed" ← Dag-version churn; only good when you know in advance the structure

We can do it better.

Introducing: a **dynamic Task Group** whose execution graph is **emitted at runtime**

Example use case: Information gathering for support tickets

Have an AI agent decide on how to process a support ticket.

Inspired and generalized from how Astronomer's own support team.

- Gather some info
- Propose deterministic checks
- Run predefined operators

DEMO

+

Home

Dags

Assets

Browse

Admin

Docs

-05:00

User

Dag

example_dynamic_task_group

manual__2026-08-29T21:14:02.068552+00:00

grid

list

graph

undo

redo

delete

search

settings

handle_ticket

- 2 Tasks

Task Group

plan

@task

generated

- 2 Tasks

Task Group

step [1]

@task

summarize

@task

React Flow

manual__2026-08-29T21:14:02.068552+00:00

Queued

refresh

check

close

trash

Logical Date	Run Type	Start Date	End Date	Duration	Triggering User
2026-08-29 16:13:54	▶ Manual				admin

Dag Version(s)
v1

add note

Task Instances

Asset Events

Audit Log

Code

Details

3 Task Instances

add filter

Task ID	Map Index	State	Start Date	Try Numbe
☐ handle_ticket.generated.su mmarize		No Status		0
☐ handle_ticket.generated.ste p		No Status		0
☐ handle_ticket.plan		No Status		0

- 
-  Home
-  Dags
-  Assets
-  Browse
-  Admin
-  Docs
-  -05:00
-  User

example_dynamic_task_group

manual__2026-08-29T21:14:02.068552+00:00

▶ Trigger

⇒ Queued

Triggering User

► Manual

v1

 Add a note...

Details

3 Task Instances

≡ Add Filter

Task ID

Map Index State Start Date 

Try Numbe

```
handle_ticket.generated.summarize
```

No Status

0

☐ `handle_ticket.generated.ste`
`p`

No Status

0

- handle_ticket.plan

No Status

0

- 
- 
Home
- 
Dags
- 
Assets
- 
Browse
- 
Admin
- 
Docs
- 
-05:00
- 
User

example_dynamic_task_group

manual__2026-08-29T21:14:02.068552+00:00

▶ Trigger



Queued

Triggering User

- ▶ Manual

v1

 Add a note...

Details

3 Task Instances

☰ Add Filter

Task ID

Map Index State 

Start Date ▾

Try Numbe

```
handle_ticket.generated.summarize
```

 No Status

0

☐ handle_ticket.generated.ste

No Status

0

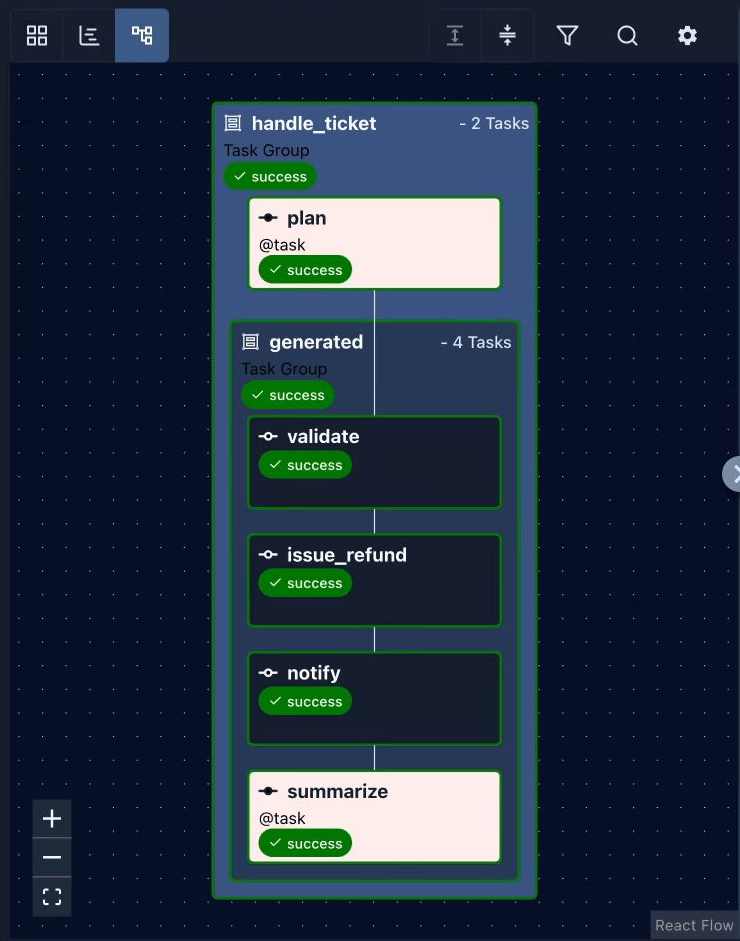
- ❑ `handle_ticket.plan`

No Status

0

Dag Run example_dynamic_task_group manual__2026-08-29T21:14:02.068552+00:00

Search Dags Trigger



manual__2026-08-29T21:14:02.068552+00:00 Success

✓ / ✕

Logical Date	Run Type	Start Date	End Date
2026-08-29 16:13:54	Manual	2026-08-29 16:16:31	2026-08-29 16:16:36
Duration	Triggering User	Dag Version(s)	
00:00:04.913	admin	v1	

Add a note...

Task Instances Asset Events Audit Log Code Details

5 Task Instances

Task ID	Map Index	State	Start Date	Try Numb
handle_ticket.generated.ste p	notify	Success	2026-08-29 16:16:34	1
handle_ticket.generated.ste p	issue_refund	Success	2026-08-29 16:16:33	1
handle_ticket.generated.su mmarize		Success	2026-08-29 16:16:35	1

YOU ALLOW THE PIECES, RUNTIME
CHOOSES THE ARRANGEMENT

How dynamic graphs can be deterministic

A key tenet of Airflow is "workflows as code."

How do we maintain that, but allow flexibility:

- Let Dags decide their tasks at runtime

e.g. pick your choice at a buffet, not a gourmet chef cooking anything you ask.

- Plan is durable, retries give you the same graph.
- Validation step before execution to impose control

Elements of Dynamic execution graph



The Catalog

list of allowable operators
defined in Dag code



The Plan

Lists of Operators+arguments, and
dependencies (nodes & edges)



The Planner

Task which generates a plan and
gives it to Airflow at runtime



The Validator

Deterministic code that runs on
worker to double check the plan

5 second syntax tour

`dynamic_task_group()` returns a `TaskGroup`

Required arguments:

- `planner` - task that plans :)
- `catalog` - list of allowed operators
- `max_selected` - number of tasks allowed
- (A few others that might change before AIP lands)



```
def triage(context) -> dict: ...

@task
def validate(**context): ...

@task
def notify(**context): ...

dynamic_task_group(
    group_id="my-dynamic-group",
    planner=triage,
    catalog=[
        validate, notify, TimeDeltaSensor, ...
    ],
    max_selected=10
)
```

Example use case: the "control table"

AKA Operations driven scheduling.

Market holiday? Flip a flag.

Re-run one region? Flip a flag.

Today in Airflow, pick your poison

- Generate the Dag file from the table → new plan= new Dag version.
- One task does it all internally → no per-job retry, no per-job logs.
- A Dag per shape → you have to enumerate the shapes first.

region	job	run?	after
EMEA	load_fx	Y	~
EMEA	revalue	Y	load_fx
APAC	load_fx	N	~
US	load_fx	Y	~
US	revalue	Y	load_fx

Not just AI

An example where dyn. graph is useful – i.e. Control-M control table

Old: dynamic dag generation (either CI pipelines, or in dag file itself) - plays badly with Dag Versioning, retries, etc.

[If this use case exists already, why didn't we solve it years ago. Be Careful. Something about expanding the use cases?]

Caveats: AIP-113 is under discussion

Proposal still being drafted

Community needs to agree on the design and constraints

If the community agrees, expect it for Airflow 3.5 in ~mid 2027

Falsehoods Airflow believed about your Dags

~~Workflows are only slowly changing~~

Dag versioning support

Since 3.0

~~Dags cannot contain cycles~~

Task group Loops

Expected in Airflow 3.4

~~Dependencies must be static and pre-defined~~

Dynamic sub-graphs

Expected in Airflow 3.5

Tell us!

Please contribute use cases

We know about a few examples here; more lets us build it better

We want to hear from you

- Where you want it
- What constraints do you need?
- How are you going to evaluate it?

The 2026 Apache Airflow[®] Survey is Live!

- Takes just 7 minutes
- Get a free AI Orchestration Fundamentals Certification Exam or DAG Authoring Certification (normally \$150 each)

